

PHYS 449 - Homework 2, Question 2

Student Name: Allen Mathew

Student ID: 20964892

Due Date: Thursday, September 18, 3:00pm

ChatGPT was used to help generate the code and some formulas for the questions

2. Produce a dataset $\mathcal{D}$ from your 16×16 Ising Monte Carlo (MC) code. Each element of the dataset is a pair of numbers: the temperature T_i and the corresponding energy $E(T_i)$ of a single MC-sampled configuration in thermal equilibrium. Choose each T_i randomly from the interval $[1.0, 1.5]$, and build a data set of size $|\mathcal{D}| = 10^3$. Divide your dataset into 80% training and 20% validation, and consider linear regression with basis functions that are polynomial in T .

Using the given Monte Carlo program for the 2D Ising model, we can easily generate a dataset of 1000 random temperatures from the interval $[1.0, 1.5]$.

```
Building dataset from equilibrium: 100%|██████████| 1000/1000  
[23:31<00:00, 1.41s/it]  
Saved dataset to ising_dataset.csv
```

We can now randomly divide the dataset into 80% training and 20% validation using the following code:

```
In [41]: import pandas as pd  
import matplotlib.pyplot as plt  
  
df = pd.read_csv("ising_dataset.csv")  
print(df.shape)  
df.head()  
  
import numpy as np  
  
rng = np.random.default_rng(42)  
idx = np.arange(len(df))  
rng.shuffle(idx)  
  
cut = int(0.8 * len(df))  
train_idx, val_idx = idx[:cut], idx[cut:]  
  
T_train, E_train = df["T"].values[train_idx], df["E"].values[train_idx]  
T_val, E_val = df["T"].values[val_idx], df["E"].values[val_idx]  
  
(1000, 2)
```

(a) Begin with a basis set up to linear order in T , and find the regression solution. Calculate the mean squared error (MSE) on both the training and the validation set. Discuss.

The regression line would take the form of:

$$E(T) \approx w_0 + w_1 T$$

Where w_0 and w_1 are the weights of the model. w_0 is the extrapolated energy at $T = 0$ and w_1 is the change in energy per spin per unit temperature (averaged over $T \in [1.0, 1.5]$). To get w , we solve the normal equation:

$$X^\top X w = X^\top y \Rightarrow w = (X^\top X)^{-1} X^\top y$$

The mean squared error (MSE) for the training data would be:

$$MSE_{training} = \frac{1}{N_{training}} \sum (E - \hat{E})^2$$

Similarly, the MSE for the validation data would be:

$$MSE_{validation} = \frac{1}{N_{validation}} \sum (E - \hat{E})^2$$

We now implement this using python:

```
In [ ]: X_train_lin = np.column_stack([np.ones_like(T_train), T_train])
X_val_lin = np.column_stack([np.ones_like(T_val), T_val])

w_lin = np.linalg.lstsq(X_train_lin, E_train, rcond=None)[0]
print("Linear weights:", w_lin)

def mse(y, yhat):
    return np.mean((y - yhat)**2)

mse_train_lin = mse(E_train, X_train_lin @ w_lin)
mse_val_lin = mse(E_val, X_val_lin @ w_lin)
print("Linear train MSE:", mse_train_lin)
print("Linear val MSE:", mse_val_lin)
```

Linear weights: [-2.02910256 0.04487042]

Linear train MSE: 0.003603171771438314

Linear val MSE: 0.004391120898844449

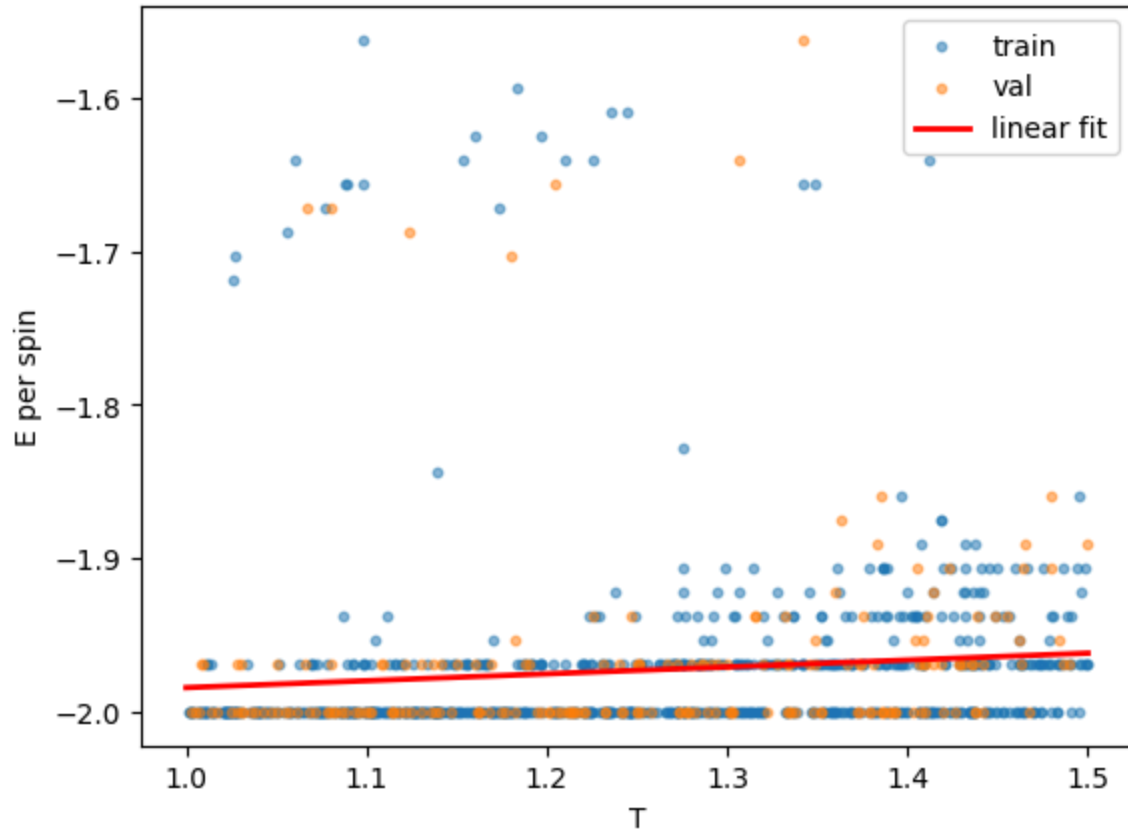
(b) Use your model to extrapolate the energy to $T = 0$. Produce a scatter plot of your datasets and the regression model. Discuss the sources of error in your zero- T prediction.

```
In [58]: Ehat_T0_lin = w_lin[0] # intercept
print("Linear extrapolation at T=0:", Ehat_T0_lin)

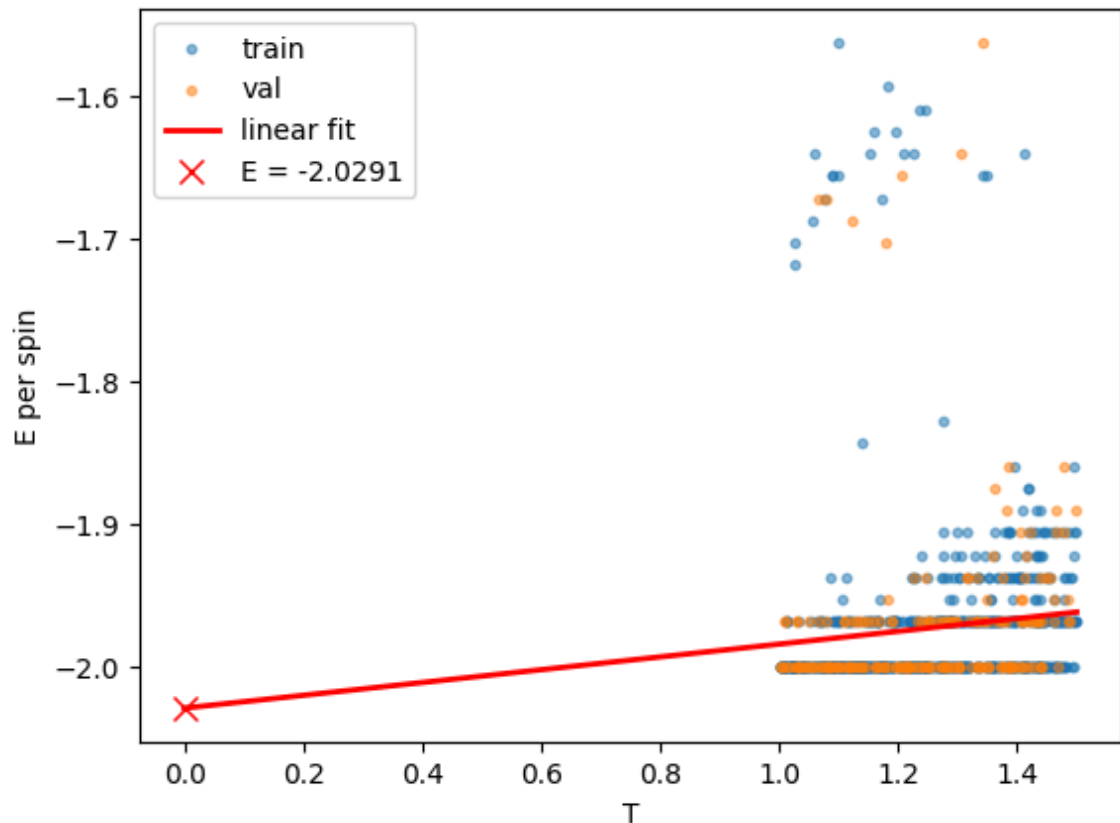
T_plot = np.linspace(1.0, 1.5, 200)
E_plot_lin = w_lin[0] + w_lin[1]*T_plot
```

```
plt.scatter(T_train, E_train, s=10, alpha=0.5, label="train")
plt.scatter(T_val, E_val, s=10, alpha=0.5, label="val")
plt.plot(T_plot, E_plot_lin, color="red", lw=2, label="linear fit")
plt.xlabel("T"); plt.ylabel("E per spin")
plt.legend(); plt.show()
```

Linear extrapolation at $T=0$: -2.0291025600002337



Using the linear regression model, we can extrapolate the energy at $T = 0$ to be ≈ -2.0291 .



According to ChatGPT, the true ground-state energy is $-2NJ$ for ferromagnetic Ising at $T = 0$. We can see that our extrapolation gave us an accurate result. The error from this model could be from the fact that we are using a linear model to model a non-linear pattern. Another error could be caused by the fact that our training data is in a very small interval of T and extrapolating points far away from this interval (like $T = 0$) can inevitably cause errors.

(c) Repeat the above steps, now including polynomial basis functions up to T^2 . Can you interpret the weights of your model?

The regression line would now take the form of:

$$E(T) \approx w_0 + w_1T + w_2T^2$$

Where w_0 is the extrapolated energy at $T = 0$, w_1 is the change in energy per spin per unit temperature at $T = 0$, and w_2 is the curvature of our training data.

By using the same method as before, we can solve this in python:

```
In [59]: X_train_quad = np.column_stack([np.ones_like(T_train), T_train, T_train**2])
X_val_quad = np.column_stack([np.ones_like(T_val), T_val, T_val**2])

w_quad = np.linalg.lstsq(X_train_quad, E_train, rcond=None)[0]
print("Quadratic weights:", w_quad)
```

```

mse_train_quad = mse(E_train, X_train_quad @ w_quad)
mse_val_quad   = mse(E_val,   X_val_quad   @ w_quad)
print("Quadratic train MSE:", mse_train_quad)
print("Quadratic val MSE:", mse_val_quad)

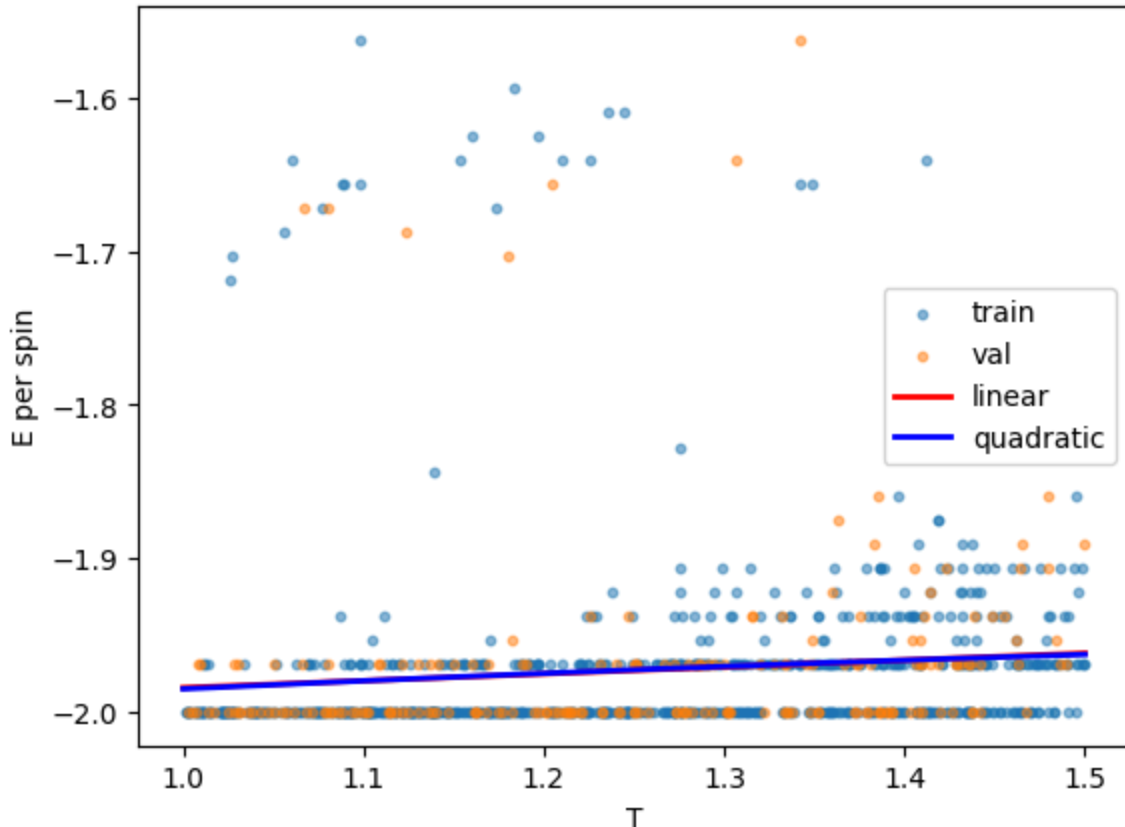
Ehat_T0_quad = w_quad[0]
print("Quadratic extrapolation at T=0:", Ehat_T0_quad)

E_plot_quad = w_quad[0] + w_quad[1]*T_plot + w_quad[2]*T_plot**2

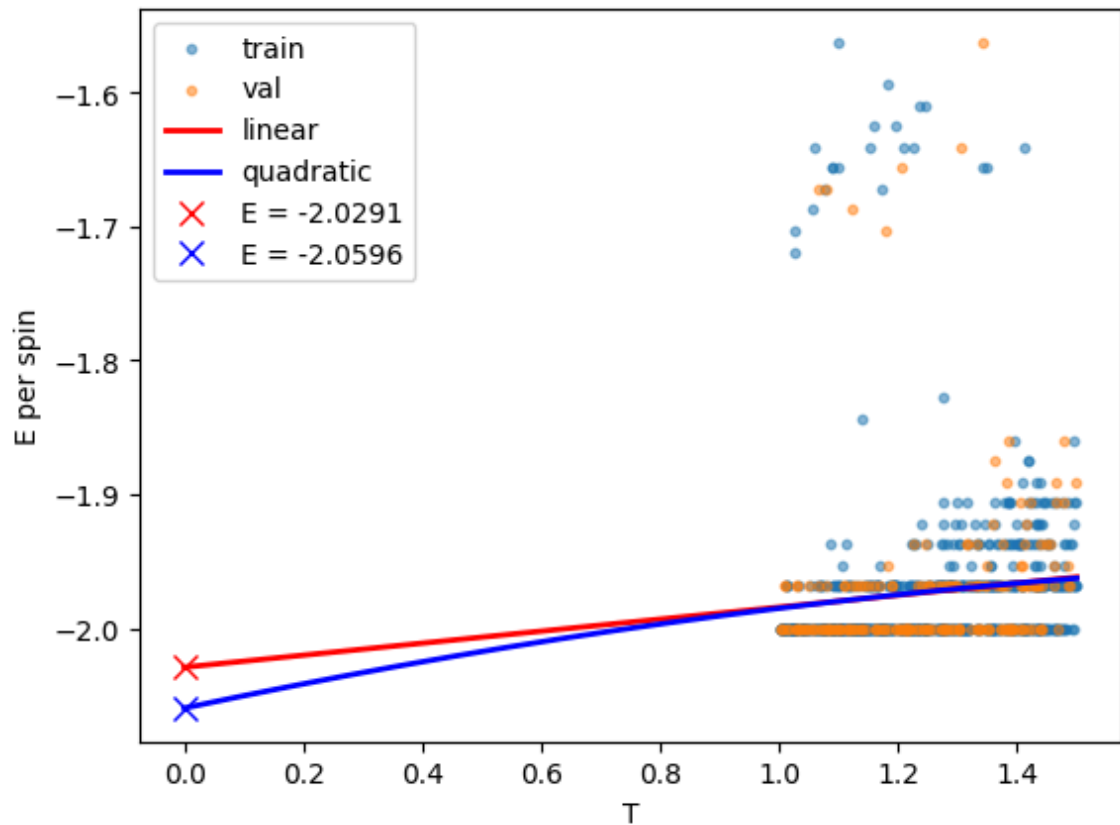
plt.scatter(T_train, E_train, s=10, alpha=0.5, label="train")
plt.scatter(T_val,   E_val,   s=10, alpha=0.5, label="val")
plt.plot(T_plot, E_plot_lin, color="red", lw=2, label="linear")
plt.plot(T_plot, E_plot_quad, color="blue", lw=2, label="quadratic")
plt.xlabel("T"); plt.ylabel("E per spin")
plt.legend(); plt.show()

```

Quadratic weights: [-2.05963852 0.09440506 -0.01982958]
 Quadratic train MSE: 0.0036030445192157014
 Quadratic val MSE: 0.004392452117384572
 Quadratic extrapolation at T=0: -2.059638518152002



One thing to notice is that the quadratic MSE for both training and validation sets are very close to the linear MSEs. It would make sense to assume that the quadratic model would have lower MSEs in general because the quadratic term can better model the curvature of the data. But, since the range of our data set is very small, we see a stronger linear relationship in our training data. Using the quadratic model, we can extrapolate the energy at $T = 0$ to be ≈ -2.0596 .



We see that the quadratic model deviates further from -2 than the linear model. Again, this error is most likely caused by the fact that the point we are extrapolating ($T = 0$) is very far away from our training data. The quadratic model works well with data in that range, but can quickly deviate from the actual value when going further from the training data.

(d) Discuss the ability of your models to generalize to unseen data. Does improved validation error within the sampled domain correlate with better predictions at $T = 0$?

We saw that the quadratic model didn't reduce MSEs much compared to the linear model, and also produced a less accurate energy value at $T = 0$. Thus, improved validation error within the sampled domain does not correlate with better predictions at $T = 0$. To get a better energy prediction, we need to increase the range of the training data so that it could be closer to $T = 0$.